

DATA VORTEX 2026 · ROUND 1 PHASE 2

SQL Queries

Team SE7EN · selected questions E3, M2, H4
Engine: MySQL 8.0 · schema: social_engine
Source tables: Phase 1 cleaned users (1,500) and posts (12,000)

What we are turning in

Three statements. They run against the cleaned Social Engine tables after 00_schema.sql and the CSV load. Nothing in the SELECT list is a typed-in number. If you load a different extract, the row counts and the 90th percentile move with it.

Schema

Two tables at their natural grain, users 1—N posts. We kept the Phase 1 helper columns (city, country, the dq_flags) because they cost nothing to store and they make a later audit possible. The queries below only need user_id, location, follower_count, platform, likes, shares and comments.

Table	Grain	Rows	Primary key	Notes
users	one account	1,500	user_id	follower_count is complete
posts	one unique post	12,000	post_id	likes and platform may be NULL

Foreign key posts.user_id → users.user_id. Every post matches. NULL likes means the source value was unrecoverable, not zero. AVG() and SUM() skip NULL on their own; we never COALESCE likes or platform to a constant.

E3 — Average Engagement by Platform

Drop posts with a missing platform (1,784 rows). Average likes, shares and comments, and the average of likes + shares + comments. ORDER BY puts the answer on row 1.

```
SELECT
  platform,
  COUNT(*)
  COUNT(likes)
  ROUND(AVG(likes), 2)
  ROUND(AVG(shares), 2)
  ROUND(AVG(comments), 2)
  ROUND(AVG(likes + shares + comments), 2)
  AS post_count,
  AS posts_with_likes,
  AS avg_likes,
  AS avg_shares,
  AS avg_comments,
  AS avg_total_engagement
FROM posts
WHERE platform IS NOT NULL
GROUP BY platform
ORDER BY avg_total_engagement DESC;
```

M2 — Do High Follower Users Get More Engagement?

Label each post by the author's follower count at the 25,000 cut the question sets, then average engagement at post grain. `vs_overall_avg` is the group mean minus the table-wide post mean so a near-zero gap is visible instead of looking like the query did nothing.

```
WITH labelled AS (
  SELECT
    u.user_id,
    u.follower_count,
    p.likes,
    p.shares,
    p.comments,
    CASE
      WHEN u.follower_count >= 25000 THEN 'High follower (>= 25,000)'
      ELSE 'Low follower (< 25,000)'
    END AS follower_group
  FROM users AS u
  INNER JOIN posts AS p
    ON p.user_id = u.user_id
)
SELECT
  follower_group,
  COUNT(DISTINCT user_id) AS user_count,
  COUNT(*) AS post_count,
  COUNT(likes) AS posts_with_engagement,
  ROUND(AVG(follower_count), 0) AS avg_followers,
  ROUND(AVG(likes), 2) AS avg_likes,
  ROUND(AVG(shares), 2) AS avg_shares,
  ROUND(AVG(comments), 2) AS avg_comments,
  ROUND(AVG(likes + shares + comments), 2) AS avg_engagement_per_post,
  ROUND(
    AVG(likes + shares + comments)
    - (SELECT AVG(likes + shares + comments) FROM posts),
    2
  ) AS vs_overall_avg
FROM labelled
GROUP BY follower_group
ORDER BY follower_group;
```

H4 — Follower to Engagement Anomaly

Collapse to one row per user, rank by total engagement with NTILE(10), keep decile 1 (top 10% of all 1,500 users), then keep follower_count < 5,000. PERCENT_RANK is only printed so the row can be read against the full user set. The filter is the window, not a typed cutoff such as total_engagement > 40000.

```
WITH user_engagement AS (
  SELECT
    u.user_id,
    u.location,
    u.follower_count,
    COUNT(p.post_id) AS post_count,
    COUNT(p.likes) AS posts_with_engagement,
    ROUND(AVG(p.likes + p.shares + p.comments), 2) AS avg_engagement,
    SUM(p.likes + p.shares + p.comments) AS total_engagement
  FROM users AS u
  INNER JOIN posts AS p
    ON p.user_id = u.user_id
  GROUP BY
    u.user_id,
    u.location,
    u.follower_count
),
ranked AS (
  SELECT
    user_engagement.*,
    NTILE(10) OVER (ORDER BY total_engagement DESC) AS engagement_decile,
    ROUND(PERCENT_RANK() OVER (ORDER BY total_engagement) * 100, 2) AS percentile_rank
  FROM user_engagement
)
SELECT
  user_id,
  location,
  follower_count,
  post_count,
  posts_with_engagement,
  avg_engagement,
  total_engagement,
  percentile_rank
FROM ranked
WHERE follower_count < 5000
  AND engagement_decile = 1
  AND total_engagement IS NOT NULL
ORDER BY total_engagement DESC;
```

How to run this in Workbench

- Execute sql/00_schema.sql.
- Import data/users_cleaned.csv into users, then data/posts_cleaned.csv into posts.
- Execute sql/02_verify_load.sql. Expect 1,500 users, 12,000 posts, 1,784 NULL platforms, 1,814 NULL likes.
- Execute this file (sql/01_queries.sql). Three Result Grid tabs.

Full click-by-click notes, including the empty-cell / NULL trap, are in MYSQL_WORKBENCH_STEPS.md in the same folder.